Modul 4

Penerapan Anti Hijacking Session Pada Aplikasi Web

Tujuan:

1. Menjelaskan konsep autentikasi, otorisasi, dan kontrol akses.
2. Memahami cara kerja JWT (Header, Payload, Signature).
3. Mengimplementasikan autentikasi sederhana menggunakan PHP.
4. Menggunakan JWT untuk otorisasi API berbasis role (Admin/User).
5. Melakukan simulasi akses endpoint API menggunakan token JWT.

Instruksi:

1. Awali setiap aktivitas anda dengan doa.
2. Pahami tujuan, dasar teori, dan latihan-latihan praktikum dengan baik.
3. Kerjakan tugas-tugas praktikum dengan baik, jujur, dan sabar.
4. Tanyakan kepada dosen dan asisten apabila ada hal-hal yang kurang jelas.

Alat yang Digunakan:

1. PC/Laptop yang mendukung instalasi dan menjalankan server lokal.
2. XAMPP/MAMP
3. Browser
4. VS Code/Nottepad++/Editor Kode lainnya
5. Modul/handout materi yang akan di praktikkan.

Dasar Teori:

1. **Konsep Session dan Cookie**
   Session dan cookie merupakan dua komponen utama dalam proses autentikasi pengguna pada aplikasi web.
   - **Session**: menyimpan data pengguna secara sementara di server selama pengguna aktif. Session biasanya memiliki *session ID* unik yang dikirimkan ke browser.
   - **Cookie**: menyimpan data kecil di sisi klien (browser) yang digunakan untuk mengidentifikasi pengguna pada kunjungan berikutnya.
2. **Konsep Session Hijacking**
   Session hijacking adalah serangan di mana penyerang mencuri/mengambil alih *session ID* milik pengguna yang sah untuk mendapatkan akses tanpa autentikasi.
   Tujuan utama → memperoleh hak akses yang sama dengan pengguna asli.
   - **Metode serangan umum**:
     • Pencurian cookie melalui serangan **XSS (Cross-Site Scripting)**.

• Penyadapan data melalui koneksi tidak aman (**HTTP sniffing**).
• Penggunaan *session fixation*, di mana penyerang memaksa korban memakai *session ID* tertentu.

3. **Teknik Pencegahan Session Hijacking**
   Untuk melindungi sistem dari ancaman session hijacking, beberapa langkah berikut perlu diterapkan:
   - **Penggunaan HTTPS (TLS)**: memastikan data terenkripsi selama transmisi sehingga *session ID* tidak dapat disadap.
   - **Regenerasi Session ID**: melakukan session_regenerate_id(true) setelah autentikasi untuk mencegah *session fixation*.
   - **Pengaturan Cookie yang Aman**: menambahkan atribut HttpOnly, Secure, dan SameSite agar cookie tidak dapat diakses skrip berbahaya.
   - **Validasi Identitas Klien**: mencatat dan memverifikasi alamat IP serta *User-Agent* pengguna pada setiap permintaan (*request*).
   - **Penerapan Timeout Session**: menetapkan waktu kedaluwarsa agar session tidak aktif terlalu lama.
   - **Pencatatan Aktivitas Login (Audit Log)**: menyimpan data seperti waktu login, IP, *User-Agent*, dan token untuk mendeteksi anomali.

4. **Implementasi Keamanan Login di PHP**
   Pada aplikasi berbasis PHP, keamanan login dapat ditingkatkan dengan:
   - Menggunakan fungsi password_hash() dan password_verify() untuk keamanan kata sandi.
   - Membatasi percobaan login untuk mencegah *brute force attack*.
   - Membuat tabel *log aktivitas* untuk memantau login dan token yang digunakan.
   - Menghapus *session* dan token saat logout atau saat terdeteksi aktivitas mencurigakan.

Praktikum

1. **Pembuatan Database dan Struktur Tabel**
   a) Buka phpMyAdmin dan buat database baru, contoh: adv_login.
   b) Jalankan SQL ini di database yang sudah dibuat

```
CREATE TABLE tb_admin
(
user VARCHAR(50) PRIMARY KEY,
     pass VARCHAR(255),
    name VARCHAR(50),
    email VARCHAR(100),
    priviledge INT(1)
);

CREATE TABLE tb_admin_log(
id INT PRIMARY KEY AUTO_INCREMENT,
    tgl DATETIME,
    expired DATETIME,
    token VARCHAR(40),
    username VARCHAR(50),
    ip VARCHAR(20),
    useragent VARCHAR(150),
    stat INT(1)
);

INSERT INTO tb_admin VALUES
('admin','$2y$10$ggxmazeAZOoioEnq4mLXSeLvn404IewZ29Ao
F7vzZwYuTRqu6AUPK',
'Administrator','admin@localhost',1);
```

2. **Koneksi dan File Library**
   a) Di folder project, buat sebuah folder conf untuk menampung file penting kita yaitu *conn.php*, *library.php*, dan *ClassLogin.php*.

```
<?php
//conf/conn.php
session_start();

$user = "root";
$pass = "";
$db = "adv_login";
$host = "localhost";

$db = new
PDO('mysql:host=localhost;dbname='.$db.';charset=utf8',$user,$pass);

require_once("conf/library.php");
require_once("conf/ClassLogin.php");
$login = new Login();
```

```
<?php
//conn/library.php

function create_alert($type, $pesan, $header=null){
 $_SESSION['adm-type'] = $type;
 $_SESSION['adm-message'] = $pesan;

 if($header!==null){
        header("location:".$header);
        exit();
 }
}

function show_alert(){
 if(isset($_SESSION['adm-type'])){
        $type = ucfirst($_SESSION['adm-type']);
        unset($_SESSION['adm-type']);
        $message = $_SESSION['adm-message'];
        unset($_SESSION['adm-message']);

        return "
        <div class='alert alert-$type'>
                <strong>$type</strong>
                <br>
                $message
        </div>
        ";
 }
}
```

```
<?php
//conn/ClassLogin.php

Class Login{
 var $db;

 public function __construct(){
        global $db;
        #menghubungkan variabel database $db ke class Login
        $this->db = $db;
 }

 public function cek_login(){
        //method yang akan memvalidasi apakah sedang dalam keadaan
log in atau tidak
        return false;
```

```
}


Public function salah_login_action($username){
        //method yang akan dipanggil apabila user memasukkan
username/password yang salah
        return false;
}

public function cek_salah_login($limit=5){
        return true;
}

public function true_login($username, $expired){
        //method yang akan dipanggil apabila user memasukkan
username dan password yang benar
        return true;
}

public function logout(){
        //method yang akan dipanggil apabila user logout dari sistem
        return true;
}

public function login_redir(){
        //method yang akan selalu dipanggil di seluruh halaman non
index dan non login,
        //untuk mengecek apabila user tidak memiliki akses langsung
diredirect ke halaman login
        if(!$this->cek_login())
                header("location:index.php");
}

}
```

b) Buat file *index.php* di direktori utama project

```
<?php
//index.php
require("conf/conn.php");

$login_status = $login->cek_login();
if($login_status){
 //bawa ke halaman home
 header("location:home.php");
 exit();
}
else{
```

```
//include form log in jika belum log in
 include "login.php";
}
?>
```

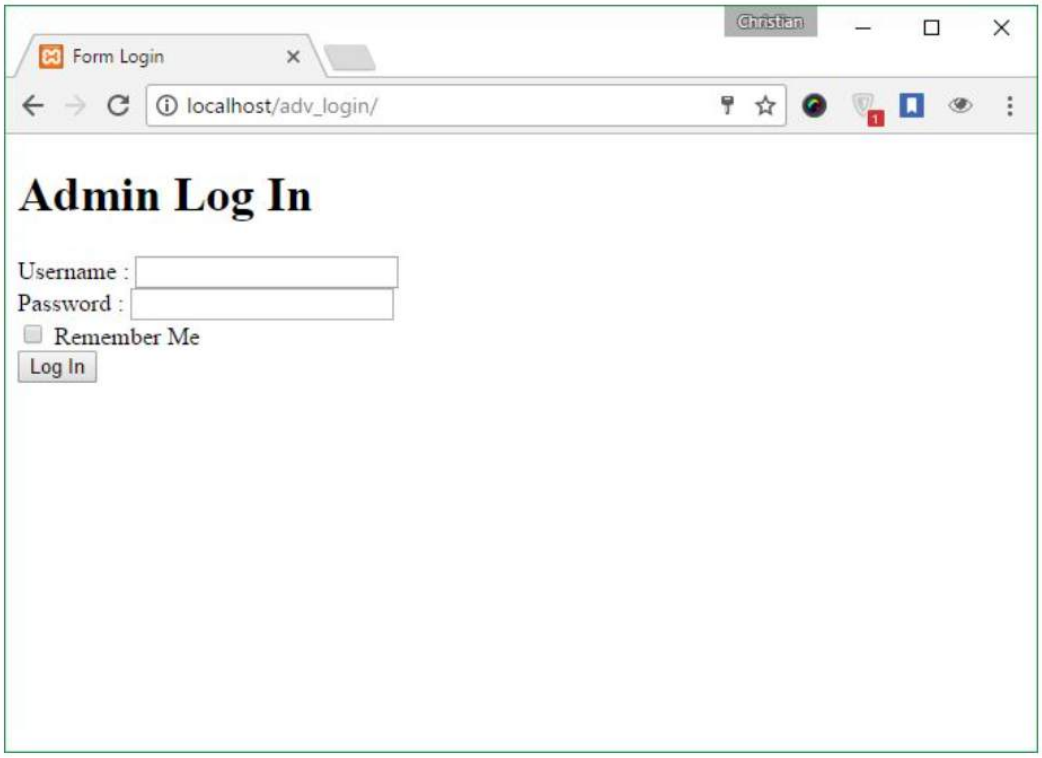
c) Buat form login di file login.php.

```
<?php
require_once("conf/conn.php");
?>
<!doctype html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Form Login</title>
</head>
<body>

<form action="login-proses.php" method="post">
 <h1>Admin Log In</h1>
 <?=show_alert();?>
 Username : <input type="text" name="username">
 <br>
 Password : <input type="password" name="password">
 <br>
 <label for="rmb">
        <input type="checkbox" name="remember" value="1"
id="rmb">
        Remember Me
 </label>
 <br>
 <button type="submit" name="btn">Log In</button>
</form>

</body>
</html>
```

d) Buka di browser, hasil decode akan terlihat seperti berikut:



3. Proses Login

a) membuat file *login-proses.php* untuk validasi login.

```
<?php
//login-proses.php
include "conf/conn.php";

if(!$login->cek_salah_login()){
 //kalau user salah login melebihi batas yang ditentukan, maka proses
langsung berhenti
 create_alert("error","Mohon maaf Anda tidak dapat login lagi karena
kesalahan login Anda terlalu banyak. Hubungi Administrator untuk
informasi lebih lanjut","index.php");
}

//tombol $_POST['btn'] harus ditekan. kalau tidak ditekan artinya nggak
ada proses apapun yang dijalankan
if(isset($_POST['btn'])){
 $username = $_POST['username'];
 $password = $_POST['password'];

 //step 1 : cek apakah username ada di tabel
 $cek = $db->query("SELECT * FROM tb_admin WHERE user = ".$db-
>quote($username));

 if($cek->rowCount() > 0){
```

```
        //username ada, tangkap password yg ada di database
        $row = $cek->fetch();
        $password_db = $row['pass'];
        #password_verify adalah fungsi PHP 5.5> yang otomatis
mengecek kesamaan inputan dengan hash
        if(password_verify($password, $password_db)){
                //password sudah cocok

                $expired = 0;
                if(isset($_POST['remember'])){
                        if($_POST['remember'] = 1){
                                $expired = '+1 year'; // 1 tahun
                        }
                }
                #kalau remember me dicentang, login akan expired dalam
waktu 1 tahun, selain itu ya akan seperti session biasa yang hilang ketika
diclose

                $login->true_login($username, $expired); //pencatatan
token akan dilakukan disini
                create_alert("success","Log In Berhasil","index.php");
        }
        else{
                //password tidak cocok
                $login->salah_login_action($username); //pencatatan
kesalahan login
                create_alert("error","Username atau password tersebut
salah","index.php");
        }

 }
 else{
        $login->salah_login_action($username); //pencatatan kesalahan
login
        create_alert("error","Username atau password tersebut tidak
terdaftar","index.php");
 }

}
```

4. Penyesuaian Class Login

a) Salin method-method dibawah ini, letakkan sesuai di class Login yang sudah ada tadi.

```
...
 public function salah_login_action($username){
        //logic : dipanggil saat user salah memasukkan
username/password.
```

```
        //username, tgl, ip, dan user agent dicatat dengan FLAG=0.

        $tgl = date("Y-m-d H:i:s");
        $ip = $_SERVER['REMOTE_ADDR'];
        $useragent = $_SERVER['HTTP_USER_AGENT'];

        //memasukkan data ke tb_admin_log dengan flag STAT = 0.
        $save = $this->db->prepare("INSERT INTO tb_admin_log
VALUES (NULL, ?, '', '', ?, ?, ?, 0)");
        $save->execute(array(
                $tgl, $username, $ip, $useragent
        ));
        return true;
 }
...
```

```
...
 public function cek_salah_login($limit=5){
        #method ini dipanggil sekali di login-proses paling atas.
        #$limit bisa disesuaikan sesuai kebutuhan kita.
        //cek apakah di tabel tb_admin_log ada 5 IP yang sama dalam
keadaan salah login (STAT = 0)

        $ip = $_SERVER['REMOTE_ADDR'];
        $cek = $this->db->prepare("SELECT * FROM tb_admin_log
WHERE stat = 0 AND ip = ?");

        $cek->execute(array($ip));
        if($cek->rowCount() >= $limit)
                return false;
        return true;
 }
...
```

```
...
 public function true_login($username, $expired){
        #method yang dipanggil ketika username dan password sudah
tepat dimasukkan

        $tgl = date("Y-m-d H:i:s");
        if($expired <> 0){
                #kalau remember me dicentang, tanggal expirenya adalah
1 tahun dari sekarang.
                $expireddb = date("Y-m-d H:i:s",strtotime($expired));
        }
        else{
```

```
                #kalau remember me tidak dicentang, secara default user
dapat login selama 6 jam saja.
                $expireddb = date("Y-m-d H:i:s",strtotime("+6 hours"));
        }

        $ip = $_SERVER['REMOTE_ADDR'];
        $useragent = $_SERVER['HTTP_USER_AGENT'];

        $token =
sha1($ip.$expireddb."string_random_apasaja".microtime()); //intinya
membuat karakter acak saja
        //$token ini penting,, nantinya akan disimpan sebagai COOKIE

        //apabila ada kesalahan login sebelumnya dengan IP & user
agent yang sama sebelumnya harus ditandai dulu
        //penandaan dilakukan dengan mengubah FLAG dari 0 menjadi 9,
sehingga di pengecekan selanjutnya data ini tidak akan dianggap
        $upd = $this->db->query("UPDATE tb_admin_log SET stat = 9
WHERE token = " AND ip = ".$this->db->quote($ip)." AND useragent =
".$this->db->quote($useragent));


        //memasukkan data lengkap ke tb_admin_log dengan flag STAT
= 1.
        $save = $this->db->prepare("INSERT INTO tb_admin_log
VALUES (NULL, ?, ?, ?, ?, ?, ?, 1)");
        $save->execute(array(
                $tgl, $expireddb, $token, $username, $ip, $useragent
        ));


        //simpan token ke cookie
        $expr = 0;
        if($expired <> 0){
                $expr = intval(strtotime($expired));
        }
        setcookie("adv_token", $token, $expr, "/");
        #kalau remember me tidak dicentang, cookie akan otomatis
bertindak sebagai session
        #kalau dicentang, cookie akan terus disimpan

        return true;
 }
...
```

```
...
 public function cek_login(){
```

```
        /*kondisi user dinyatakan login adalah :
        1. Memiliki $_COOKIE['adv_token']; (yang dibuat di method
true_login() tadi)
        2. $_COOKIE['adv_token'] terdaftar di tabel tb_admin_log, dan
dalam keadaan masih belum expired
        3. IP dan User Agent sesuai dengan token yang terdaftar
        */
        if(isset($_COOKIE['adv_token'])){
                $token = $_COOKIE['adv_token'];
                $now = date("Y-m-d H:i:s");
                $cek = $this->db->query("SELECT * FROM tb_admin_log
WHERE token = ".$this->db->quote($token)." AND expired > ".$this-
>db->quote($now));
                if($cek){
                        #kalau token di cookie tersebut ada, lakukan
pengecekan IP dan User Agent
                        $row = $cek->fetch();
                        if($row['ip'] == $_SERVER['REMOTE_ADDR'] ||
$row['useragent'] == $_SERVER['HTTP_USER_AGENT']){
                                //kondisi bisa disesuaikan utk kebutuhan
dengan ATAU / DAN
                                //kondisi DAN boleh dipakai, tapi terlalu
strict.. Lebih baik pakai ATAU saja.
                                $username = $row['username'];

                                //kembalikan data user yg sedang login,,
siapa tahu nanti ingin diolah
                                $get_admin = $this->db->query("SELECT *
FROM tb_admin WHERE user = ".$this->db->quote($username));
                                $rget = $get_admin->fetch();

                                return array(
                                        "username" => $rget['user'],
                                        "name" => $rget['name'],
                                        "email" => $rget['email'],
                                        "priviledge" => $rget['priviledge']
                                );

                        }

                }
        }
        return false;
}
...
```

```
...
 public function logout(){
        #dipanggil saat user logout dari sistem.

        if(isset($_COOKIE['adv_token'])){
                $token = $_COOKIE['adv_token'];

                //cara menghapus cookie adalah dengan mengubah
tanggal expirednya menjadi sekarang
                $now = date("Y-m-d H:i:s");
                unset($_COOKIE['adv_token']);
                setcookie("adv_token",null,$now,"/");

                #jangan lupa tanggal expired di database diupdate juga,
supaya session token yang sudah logout tidak dihijack
                $this->db->query("UPDATE tb_admin_log SET expired =
".$this->db->quote($now)." WHERE token = ".$this->db-
>quote($token));
        }

        return true;
 }
...
```

5. Pengujian

a) Buat file tambahan di direktori utama : home.php

```
<?php
include "conf/conn.php";
$login->login_redir(); //ini method untuk mencegah user membypass
halaman dengan akses langsung ke URL
?>
<!doctype html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Contoh Halaman</title>
</head>
<body>
 <ul>
        <li><a href="menu1.php">Menu 1</a></li>
        <li><a href="menu2.php">Menu 2</a></li>
        <li><a href="menu3.php">Menu 3</a></li>
        <li><a href="logout.php">Logout</a></li>
 </ul>
```

```
 <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Explicabo, autem, dolor, vitae inventore natus consectetur accusamus officiis veritatis illo quasi molestiae iusto nesciunt officia rem omnis culpa reiciendis odio harum eaque assumenda at iure dolorum!</p>
</body>
</html>
```

b) Buat file logout di direktori utama : logout.php.

```
<?php
require("conf/conn.php");

$login->logout();
create_alert("Success","Anda sudah logout dari sistem","index.php");
```

6. Hasil

Pertama, metode Bruteforce tidak akan berjalan di halaman Log In ini, karena sudah ada fungsi cek_salah_login() yang akan menutup akses login ketika sudah mencapai sekian kali kesalahan.

Kedua, metode SQL Injection juga nggak akan berjalan, karena menggunakan escaping ($db->quote), dan fungsi prepared statement di semua query yang ada.

Ketiga, metode Session Hijacking juga tidak akan berjalan, karena cookie token yang diremember sekalipun disimpan untuk dicek IP dan User Agentnya.

Sehingga apabila cookie tersebut dicuri sekalipun, karena IP dan User Agent yang berbeda login tidak dapat dilakukan juga.